GDS-ParT-MoE

A per-particle mixture of experts for the Gated Dual-Stream Particle Transformer
Architecture, mathematics, training and first expert-specialisation results

Avinash Chandra Gupta
https://github.com/Avinashistoocool/GDS-parT-MoE

Version 1.0.0 October 2026

Abstract

GDS-ParT-MoE replaces the particle-level feed-forward networks of the Gated Dual-Stream Particle Transformer (GDS-ParT) with a token-choice mixture of experts. Each real jet constituent is routed to the top $k = 2$ of $E = 8$ expert FFNs, and only padding is excluded. Because the router sees one particle at a time, its decisions can be compared directly with that particle's physics: species, charge, momentum share, angle to the jet axis, transverse impact parameter, and the Lund-plane variables of its nearest neighbour. The model is built by sparse upcycling from a trained dense GDS-ParT checkpoint, which makes the untrained MoE reproduce the dense network exactly. This report documents the motivation, the architecture, every equation used by the code (router, renormalised top-k mixture, Switch-style load-balancing loss, router z-loss, upcycling, optimisation schedule and evaluation metrics), and the expert-analysis toolkit (routing statistics, physics profiles, normalised mutual information and expert knockout). It closes with the first, preliminary result: after a single fine-tuning epoch on 300k JetClass jets the 8-expert model reaches validation accuracy 0.8472 (dense: 0.8487), and its first-layer router already partitions particles by charge and species (normalised mutual information 0.45 and 0.43), while deeper layers remain close to uniform.

Contents

1 Motivation **2**

2 Background: the dense GDS-ParT block **2**

3 Architecture **3**
3.1 Token selection . . . 3
3.2 Router . . . 3
3.3 Top-k selection and renormalised mixture . . . 4
3.4 Parameter count . . . 5

4 Auxiliary losses **5**
4.1 Switch load-balancing loss . . . 5
4.2 Router z-loss . . . 5
4.3 Total objective . . . 5

5 Sparse upcycling **6**

6 Training **6**

7 Metrics **7**

8 Expert analysis **7**
8.1 Recorded routing . . . 8
8.2 Physics variables . . . 8
8.3 Routing statistics . . . 8
8.4 Normalised mutual information . . . 8
8.5 Expert knockout . . . 9

9 Results **9**
9.1 Run . . . 9
9.2 Mutual information across layers . . . 9
9.3 Layer-0 experts . . . 10
9.4 Knockout . . . 11

10 Testing **11**

11 Limitations **12**

12 Outlook **12**

Source map **13**

1 Motivation

Jet-tagging transformers such as ParT and GDS-ParT classify a jet from its constituents with high accuracy, but it is hard to say *what* a given layer has learned. Attention maps are one handle, yet they describe pairs, depend on the value vectors and later layers, and are difficult to relate to a single physical property.

A mixture of experts (MoE) offers a different handle. If the feed-forward part of each block is split into several experts and a router decides, for every particle separately, which experts process it, then the routing decision is a discrete, per-particle label that the network itself chose. One can then ask simple questions:

- Does an expert handle one particle species (photons, neutral hadrons, leptons)?
- Does it handle the hard core of the jet, or soft wide-angle radiation?
- Does it handle displaced tracks, as produced by b and c hadron decays?
- Does it handle collinear splittings, i.e. a particular region of the Lund plane?
- Which jet classes rely on it, in the sense that removing it hurts their accuracy?

If the experts specialise, the routing table is a learned partition of particle phase space. If they do not, the router entropy and the mutual-information numbers say so directly. The project was carried out under the supervision of Prof. Abhishek Iyer between May and July 2026.

The design goal was to change as little as possible of GDS-ParT. Attention, the relational query/key gates, the additive pairwise bias, the class-attention (CLS) blocks and the classifier are the original modules from `gds_part_colab.py`. Only the particle FFNs become MoE layers.

2 Background: the dense GDS-ParT block

The notation follows the GDS-ParT mathematical notes. A jet has N constituent slots with padding mask $m_i \in \{0, 1\}$ (1 for a real particle). Each particle carries 17 normalised features $f_i \in \mathbb{R}^{17}$, and each pair carries the four ParT interaction features

$$U_{ij} = (\ln \Delta_{ij},\ \ln k_{T,ij},\ \ln z_{ij},\ \ln m_{ij}^2), \tag{1}$$

computed on the GPU from the constituent four-vectors (`data/jetclass_cache.py`).

The model width is $D = 128$ with $H = 8$ heads, $L = 8$ particle blocks, FFN hidden width $D_{\text{ff}} = 512$, pair-embedding width $D_p = 64$, dropout 0.1 and at most 128 particles. Particle and pair embeddings are

$$x_i^{(0)} = \text{Drop}\left(W_2\,\text{GELU}(W_1\,\text{LN}(f_i) + b_1) + b_2\right) \in \mathbb{R}^D, \tag{2}$$

$$r_{ij} = W_4\,\text{GELU}(W_3 U_{ij} + b_3) + b_4 \in \mathbb{R}^{D_p}. \tag{3}$$

In the checkpoint used here (`v2_shared`) the relational gate is shared by all blocks and uses a masked mean over neighbours,

$$s_i = \frac{\sum_j m_j r_{ij}}{\max(1, \sum_j m_j)}, \qquad g_i^a = 1 + \tanh\big(\text{MLP}_a(s_i)\big), \quad a \in \{Q, K\}, \tag{4}$$

with the last layer of each gate MLP zero-initialised so that $g = 1$ at initialisation. A per-head

additive pair bias $\mathcal{B}_{ij}^{(h)} = W_{\text{bias}} r_{ij} + b_{\text{bias}}$ is shared by all blocks. One dense pre-LN block ℓ computes

$$h_i = \mathrm{LN}_1(x_i), \tag{5}$$

$$q'_i = (W_Q h_i + b_Q) \odot g_i^Q, \qquad k'_i = (W_K h_i + b_K) \odot g_i^K, \qquad v_i = W_V h_i + b_V, \tag{6}$$

$$A_{ij}^{(h)} = \mathrm{softmax}_j\Big(q_i'^{(h)\top} k_j'^{(h)} / \sqrt{d} + \mathcal{B}_{ij}^{(h)} + \log m_j\Big), \qquad d = D/H = 16, \tag{7}$$

$$\tilde{x}_i = x_i + W_O \,\mathrm{concat}_h\Big(\sum_j A_{ij}^{(h)} v_j^{(h)}\Big) + b_O, \tag{8}$$

$$x_i^{\text{next}} = \tilde{x}_i + \mathrm{FFN}(\mathrm{LN}_2(\tilde{x}_i)), \tag{9}$$

where $\log m_j$ is 0 for real particles and $-\infty$ for padding, and

$$\mathrm{FFN}(u) = \mathrm{Drop}\left(W^{\text{out}}\,\mathrm{Drop}(\mathrm{GELU}(W^{\text{in}} u + b^{\text{in}})) + b^{\text{out}}\right), \qquad W^{\text{in}} \in \mathbb{R}^{512\times 128},\ W^{\text{out}} \in \mathbb{R}^{128\times 512}. \tag{10}$$

After the eight particle blocks, two class-attention blocks update a learned CLS token, and a two-layer classifier maps $\mathrm{LN}(c)$ to ten logits. The MoE extension touches only equation (9).

3 Architecture

Figure 1 shows the model. The code lives in `moe.py`:

`GDSParTMoEConfig`
extends `GDSParTConfig` with `n_experts` (E, default 8), `top_k` (k, default 2), `moe_layers` (indices of blocks that get an MoE; default all eight), `aux_weight` ($\lambda_{\text{bal}} = 10^{-2}$) and `z_weight` ($\lambda_z = 10^{-3}$).

`MoEFFN`
the router and the expert list. Each expert is the same `_ffn(cfg)` module as the dense FFN ($128 \to 512 \to 128$ with GELU and dropout).

`GDSParTMoEBlock`
the dense block with `self.ffn` replaced by `MoEFFN` if the block index is in `moe_layers`. It returns the new state and a two-vector of auxiliary losses (zero for non-MoE blocks).

`GDSParTMoE`
the dense model with these blocks. It sums the auxiliary losses of all blocks, stores the weighted sum in `self.aux_loss`, and adds helpers for recording routing (`set_record`) and knocking out experts (`disable_experts`).

`upcycle_state_dict`
converts a dense checkpoint into MoE weights (Section 5).

3.1 Token selection

Only real particles are routed. The layer gathers the $T = \sum_{b,i} m_{bi}$ real tokens of the micro-batch into a matrix $u \in \mathbb{R}^{T\times D}$ (`x[mask]`), routes them, and scatters the outputs back. Padding slots receive an exact zero FFN output and never enter the loss statistics. Without a mask, all BN slots are treated as real.

3.2 Router

For a token $u_i = \mathrm{LN}_2(\tilde{x}_i) \in \mathbb{R}^D$ the router is a bias-free linear map

$$z_i = W_r u_i \in \mathbb{R}^E, \qquad W_r \in \mathbb{R}^{E\times D}, \qquad (W_r)_{ej} \sim \mathcal{N}(0, 0.02^2) \text{ at initialisation.} \tag{11}$$

The logits are cast to float32 even under mixed precision. Disabled experts (knockout, Section 8.5) get $z_{ie} = -\infty$. Routing probabilities are

$$\pi_{ie} = \frac{\exp z_{ie}}{\sum_{a=1}^{E} \exp z_{ia}}. \tag{12}$$

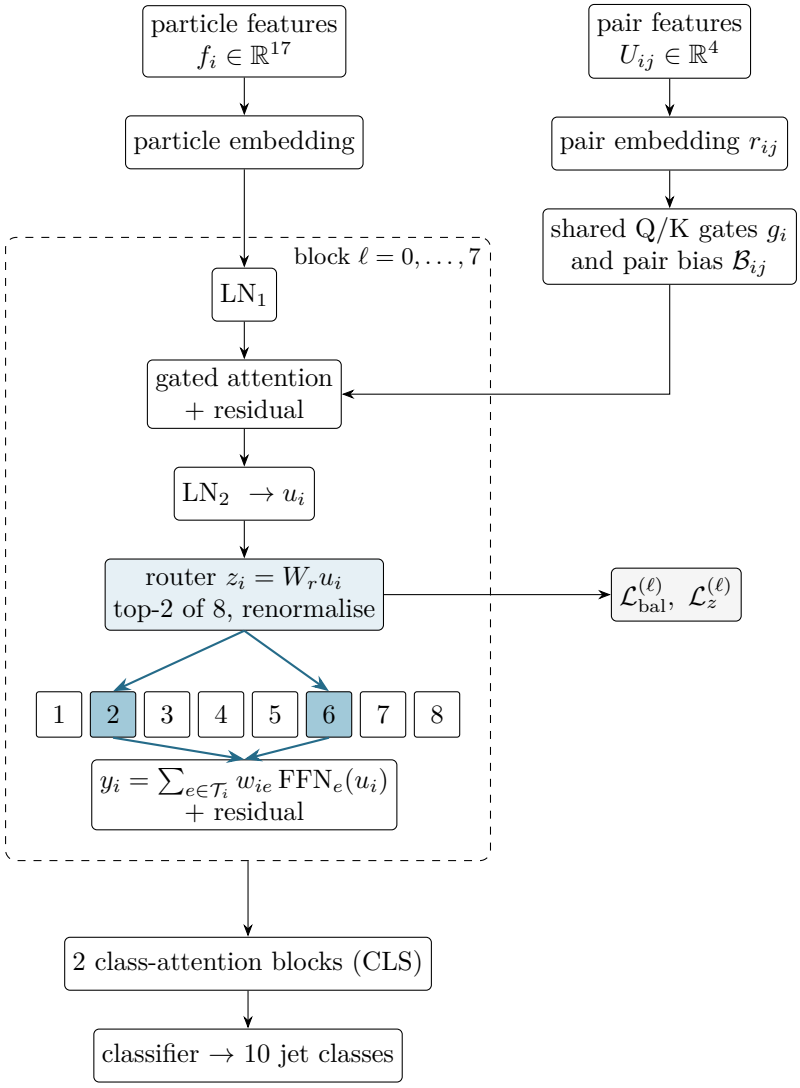


Figure 1: GDS-ParT-MoE. Inside every particle block the FFN is replaced by a router and eight expert FFNs. Each real particle i is sent to its two highest-probability experts (shaded), whose outputs are mixed with renormalised weights. Attention, gates, pair bias and CLS pooling are the unchanged GDS-ParT modules. Each MoE layer also returns a load-balancing and a z-loss term.

3.3 Top-k selection and renormalised mixture

Let $\mathcal{T}_i = \mathrm{TopK}(\pi_i, k)$ be the indices of the k largest probabilities. The mixing weights are the selected probabilities renormalised to sum to one,

$$w_{ie} = \frac{\pi_{ie}}{\max\left(10^{-9},\ \sum_{a \in \mathcal{T}_i} \pi_{ia}\right)}, \qquad e \in \mathcal{T}_i, \tag{13}$$

and the MoE output is

$$y_i = \sum_{e \in \mathcal{T}_i} w_{ie}\ \mathrm{FFN}_e(u_i), \qquad x_i^{\mathrm{next}} = \tilde{x}_i + y_i. \tag{14}$$

In code the experts are visited one at a time; expert e processes only the rows routed to it and its weighted output is accumulated with `index_add_`. There is no capacity limit and no token dropping: every real particle is processed by exactly k experts, however unbalanced the routing is. The cost of an imbalanced router is therefore compute and specialisation, never lost tokens.

Why $k = 2$ by default. With $k \geq 2$ the task loss reaches the router through the weights, for example with $\mathcal{T}_i = \{a, b\}$,

$$w_{ia} = \frac{\pi_{ia}}{\pi_{ia} + \pi_{ib}} = \sigma(z_{ia} - z_{ib}), \qquad \frac{\partial y_i}{\partial z_{ia}} = w_{ia} w_{ib} \big(\mathrm{FFN}_a(u_i) - \mathrm{FFN}_b(u_i) \big). \tag{15}$$

With $k = 1$ the single weight is identically one, so the task loss gives the router no gradient away from selection boundaries; only the auxiliary losses train it. The `-top_k 1` option is still useful for reading routing, but it is not a free improvement.

3.4 Parameter count

The $D \to D_{\mathrm{ff}} \to D$ expert has $128 \cdot 512 + 512 + 512 \cdot 128 + 128 = 131{,}712$ parameters and each router has $8 \cdot 128 = 1{,}024$. With all eight blocks converted, the MoE adds $8\,(7 \cdot 131{,}712 + 1{,}024) = 7{,}384{,}064$ parameters to the dense model's 2,074,420, for 9,458,484 in total (this matches `n_params` in the training history). Counting only two experts per layer, roughly 3,136,308 parameters act on a given particle. This is a per-token figure; a batch generally touches all experts.

4 Auxiliary losses

Let T be the number of real tokens in the micro-batch, $\mathbb{1}[\cdot]$ the indicator and $\mathcal{A} \subseteq \{1, \ldots, E\}$ the set of live (not knocked out) experts, $E_{\mathrm{live}} = |\mathcal{A}|$. During training $\mathcal{A}$ is all experts and $E_{\mathrm{live}} = E = 8$.

4.1 Switch load-balancing loss

The dispatch fraction and the mean router probability of expert e are

$$f_e = \frac{1}{Tk} \sum_{i=1}^{T} \mathbb{1}[e \in \mathcal{T}_i], \qquad \bar{\pi}_e = \frac{1}{T} \sum_{i=1}^{T} \pi_{ie}, \tag{16}$$

and the balance loss is

$$\mathcal{L}_{\mathrm{bal}} = E_{\mathrm{live}} \sum_{e \in \mathcal{A}} f_e \, \bar{\pi}_e. \tag{17}$$

Both f and $\bar{\pi}$ sum to one. For a perfectly uniform router $f_e = \bar{\pi}_e = 1/E$ and $\mathcal{L}_{\mathrm{bal}} = 1$; concentrating load on a few experts raises it. The indicator is not differentiable, so the gradient flows through $\bar{\pi}_e$: experts that already receive many tokens get their probabilities pushed down.

4.2 Router z-loss

$$\mathcal{L}_z = \frac{1}{T} \sum_{i=1}^{T} \Big(\log \sum_{e=1}^{E} \exp \tilde{z}_{ie} \Big)^2, \qquad \tilde{z}_{ie} = \begin{cases} z_{ie} & e \in \mathcal{A} \\ 0 & e \notin \mathcal{A} \end{cases}. \tag{18}$$

It keeps the router logits small, which keeps the softmax away from saturation and helps numerical stability under float16 autocast. (Replacing knocked-out logits by zero only avoids an infinite value during analysis; it does not affect training.)

4.3 Total objective

Each block returns $(\mathcal{L}_{\mathrm{bal}}^{(\ell)}, \mathcal{L}_z^{(\ell)})$, zero for non-MoE blocks. The model stores

$$\mathcal{L}_{\mathrm{aux}} = \lambda_{\mathrm{bal}} \sum_{\ell} \mathcal{L}_{\mathrm{bal}}^{(\ell)} + \lambda_z \sum_{\ell} \mathcal{L}_z^{(\ell)}, \qquad \lambda_{\mathrm{bal}} = 10^{-2}, \ \lambda_z = 10^{-3}, \tag{19}$$

summed (not averaged) over layers, and the training loss for a micro-batch is

$$\mathcal{L} = \mathrm{CE}_\varepsilon(\ell, y) + \mathcal{L}_{\mathrm{aux}}, \qquad \mathrm{CE}_\varepsilon = -(1 - \varepsilon) \log p_y - \frac{\varepsilon}{C} \sum_{c=1}^{C} \log p_c, \tag{20}$$

with label smoothing $\varepsilon = 0.1$ and $C = 10$ classes. With eight uniform MoE layers the balance term contributes about $8 \times 10^{-2} \times 1 = 0.08$ to the loss. Lowering `-aux_weight` trades balance for sharper specialisation.

5 Sparse upcycling

Training an MoE from scratch on JetClass is expensive. Instead, `-init_dense` loads a trained dense GDS-ParT checkpoint and copies it into the MoE (`upcycle_state_dict`):

1. Every non-FFN tensor (embeddings, gates, attention, pair bias, CLS blocks, classifier) is copied unchanged.
2. For each MoE block ℓ, every expert receives a copy of that block's dense FFN weights. With `-upcycle_noise` σ (default 0.01), each weight matrix (tensors with more than one dimension) is perturbed,

$$W^{(e)} = W + \sigma \, \mathrm{std}(W) \, \xi^{(e)}, \qquad \xi^{(e)}_{ab} \sim \mathcal{N}(0, 1), \tag{21}$$

   while biases are copied exactly. The noise breaks the symmetry between experts so that they can diverge.
3. Routers keep their fresh $\mathcal{N}(0, 0.02^2)$ initialisation.

Exact equivalence for $\sigma = 0$. If every expert equals the dense FFN f, then for any router and any top-k set

$$y_i = \sum_{e \in \mathcal{T}_i} w_{ie} f(u_i) = f(u_i) \sum_{e \in \mathcal{T}_i} w_{ie} = f(u_i), \tag{22}$$

because the renormalised weights sum to one. The untrained MoE therefore reproduces the dense model in evaluation mode, up to floating-point rounding. The unit test `test_upcycle_matches_dense` checks this to 10^{-5} on random inputs after perturbing all dense parameters (so that the identity-initialised gates are not trivially one). With $\sigma = 0.01$ the equivalence is approximate; README reports validation accuracy 0.8479 for the upcycled model before fine-tuning against 0.8487 for the dense checkpoint.

6 Training

`train_moe.py` trains on the memory-mapped JetClass cache built by the GDS-ParT trainer. The cache stores float16 particle features and float32 four-vectors in ragged form; batches are padded only to their longest jet, and training batches are formed from pools of jets bucketed by multiplicity to reduce the $O(N^2)$ pair cost. The pair features U_{ij} are computed on the GPU per batch.

Optimiser. The parameters are split into two AdamW groups: router weights (any name containing `.router.`) with learning rate 5η, and everything else with $\eta = 2 \times 10^{-4}$. The routers are new, while the rest of the network is already trained, so the routers are allowed to move faster.

Schedule. With S total optimiser steps and $S_w = \lfloor 0.03S \rfloor$ warm-up steps, the learning-rate multiplier at step s is

$$\rho(s) = \begin{cases} s / \max(1, S_w) & s < S_w, \\ \frac{1}{2}\Big(1 + \cos\big(\pi \min\big(1, \frac{s - S_w}{\max(1, S - S_w)}\big)\big)\Big) & s \geq S_w. \end{cases} \tag{23}$$

Setting	Flag	Default
Experts / top-k / MoE blocks	`-n_experts`, `-top_k`, `-moe_layers`	8 / 2 / all
Balance and z weights	`-aux_weight`, `-z_weight`	10^{-2}, 10^{-3}
Upcycling noise	`-upcycle_noise`	0.01
Optimiser	AdamW	lr 2×10^{-4}, weight decay 10^{-2}
Router learning-rate multiplier	`-router_lr_mult`	5 (router lr 10^{-3})
Warm-up / schedule	`-warmup_frac`	3 % linear, then cosine to 0
Gradient clipping	`-grad_clip`	global norm 1.0
Label smoothing	`-label_smoothing`	0.1
Batch size	`-batch_size`	512 jets
Pair budget per micro-batch	`-max_pairs_per_step`	2×10^6
Precision	`-amp`	float16 autocast with GradScaler
Epochs	`-epochs`	3

Table 1: Training settings in `train_moe.py`.

Micro-batching. A batch of B jets padded to N particles is split into micro-batches of $\max(1, \min(B, \lfloor 2 \times 10^6 / N^2 \rfloor))$ jets so that the $B \times N \times N$ pair tensors fit in memory. Each micro-batch loss is scaled by its share b/B of the batch before the backward pass, so the accumulated gradient is that of the batch-mean loss. The auxiliary losses are computed per micro-batch, i.e. f_e and $\bar{\pi}_e$ are micro-batch statistics. In training mode each block is also wrapped in activation checkpointing.

Evaluation and checkpoints. After each epoch the model is evaluated on the validation cache (at most `-val_max_jets`, default 100,000, drawn with a fixed seed) and `best.pt` and `last.pt` are written together with `history.json`.

7 Metrics

`metrics.py` follows the definitions of the ParT paper. For softmax scores $p \in \mathbb{R}^{n \times C}$ and labels y:

- **Accuracy**: $\frac{1}{n} \sum_j \mathbb{1}[\arg\max_c p_{jc} = y_j]$.
- **Macro one-vs-one AUC**: the scikit-learn `roc_auc_score` with `average="macro"`, `multi_class="ovo"`.
- **Background rejection**: for each signal class S against QCD, restrict to jets of those two classes and use the two-class score

$$t_j = \frac{p_{jS}}{p_{jS} + p_{j,\mathrm{QCD}} + 10^{-12}}. \tag{24}$$

  The false-positive rate at a target signal efficiency (true-positive rate) ϵ_S is read off the ROC curve by linear interpolation, and $\mathrm{Rej}_{\epsilon_S} = 1/\mathrm{FPR}$. The targets are $\epsilon_S = 50\,\%$ for Hbb, Hcc, Hgg, H4q, Tbqq, Wqq and Zqq, 99 % for Hqql and 99.5 % for Tbl.
- **Per-class accuracy**: $\Pr(\arg\max p = c \mid y = c)$, also used by the knockout study.

8 Expert analysis

`analyze_experts.py` loads a checkpoint, runs it in evaluation mode on validation jets with routing recording switched on, and relates the routing of every real particle in every MoE layer to physics quantities.

8.1 Recorded routing

For particle i in MoE layer ℓ the script keeps the full probability vector $\pi_i^{(\ell)}$ and the top-1 expert $t_i^{(\ell)} = \arg\max_e \pi_{ie}^{(\ell)}$ (the first of the top-k indices). All statistics below use $t_i^{(\ell)}$.

8.2 Physics variables

The cached features are normalised, so the script inverts the normalisation:

$$\text{type}_i = \arg\max(f_{i,8}, \ldots, f_{i,12}) \in \{\text{ch. had}, \text{neu. had}, \gamma, e, \mu\}, \tag{25}$$

$$q_i = \text{round}(f_{i,7}), \qquad \ln p_{T,i} = f_{i,0}/0.7 + 1.7, \tag{26}$$

$$z_i = p_{T,i}/p_{T,\text{jet}} = \exp(f_{i,2}/0.7 - 4.7), \qquad \Delta R_i = f_{i,4}/4 + 0.2, \tag{27}$$

$$|\tanh d_0|_i = |f_{i,13}|. \tag{28}$$

These are exact inverses only inside the $[-5, 5]$ clipping range of the cache and up to float16 rounding. From the four-vectors it also computes, with rapidity y and wrapped azimuth difference,

$$\Delta_{ij} = \sqrt{(y_i - y_j)^2 + \delta\phi_{ij}^2}, \qquad j^*(i) = \arg\min_{j \neq i,\ m_j = 1} \Delta_{ij}, \tag{29}$$

$$\Delta R_i^{\text{nn}} = \Delta_{ij^*}, \qquad \ln k_{T,i}^{\text{nn}} = \ln\max\left(10^{-4},\ \min(p_{T,i}, p_{T,j^*})\,\Delta R_i^{\text{nn}}\right), \tag{30}$$

the nearest-neighbour distance and the Lund-plane k_T of that closest pair, and the p_T rank of the particle inside its jet (0 for the hardest). Each particle also carries its jet's truth class and multiplicity.

8.3 Routing statistics

For each layer, with n recorded particles:

$$\text{load}_e = \frac{1}{n}\sum_i \mathbb{1}[t_i = e], \tag{31}$$

$$\bar{H} = \frac{1}{n}\sum_i \Big(-\sum_e \pi_{ie} \ln(\pi_{ie} + 10^{-9}) \Big), \qquad \bar{H} \leq \ln E = \ln 8 \approx 2.08 \text{ nats}, \tag{32}$$

$$P(e \mid \text{type} = \tau) = \frac{\sum_i \mathbb{1}[t_i = e,\ \text{type}_i = \tau]}{\sum_i \mathbb{1}[\text{type}_i = \tau]}, \qquad P(e \mid \text{class} = c) \text{ analogously}, \tag{33}$$

$$\text{enrich}_{e,\tau} = P(e \mid \tau)/\text{load}_e. \tag{34}$$

The per-expert *physics profile* is the mean of each variable over the particles whose top-1 expert is e, plus the charged fraction $\Pr(q \neq 0 \mid t = e)$. The plain-language report flags a variable as high or low for an expert when its profile value is more than one standard deviation (across the eight experts) above or below the expert average.

8.4 Normalised mutual information

For the expert label t and a discrete variable v,

$$\text{NMI}(t; v) = \frac{I(t; v)}{\min\left(H(t), H(v)\right)}, \qquad I(t; v) = \sum_{a,b} p(a, b) \ln \frac{p(a, b)}{p(a)p(b)}, \tag{35}$$

computed from the empirical joint histogram. Particle type, charge (shifted to $\{0, 1, 2\}$) and jet class are used directly; each continuous variable is cut into ten quantile bins (deciles, merged where edges coincide). NMI $= 0$ means the routing carries no information about v; NMI $= 1$ means one determines the other up to the smaller entropy.

8.5 Expert knockout

`disable_experts` sets the logits of chosen experts to $-\infty$. The softmax then renormalises over the live experts and the top-k is taken among them, so each affected particle is rerouted rather than dropped. For each layer ℓ and expert e,

$$\Delta\mathrm{acc}_c(\ell, e) = \mathrm{acc}_c(\text{model with } e \text{ removed in } \ell) - \mathrm{acc}_c(\text{model}), \tag{36}$$

measured on the first `-knockout_jets` validation jets. A negative value means class c depends on that expert.

9 Results

All numbers in this section are taken from files in the repository: `checkpoints/moe8_short/history.json` (training run, not tracked by git), `analysis/moe8_short/report.json` and `REPORT.md`, and the README.

9.1 Run

The dense `v2_shared_10M_10ep` GDS-ParT checkpoint (shared identity gates, pair bias, two CLS blocks; 84.8 % test accuracy and AUC 0.985 on 1M JetClass test jets according to the README) was upcycled to 8 experts with top-2 routing in all eight blocks, with $\sigma = 0.01$, $\lambda_{\mathrm{bal}} = 10^{-2}$, $\lambda_z = 10^{-3}$ and the defaults of Table 1. It was fine-tuned for one epoch on the `train_train_10M_30000_128` cache (300k jets) and validated on 20,000 jets.

Model	Validation accuracy	Validation AUC (macro OvO)
Dense GDS-ParT (`v2_shared`)	0.8487	–
Upcycled MoE, before fine-tuning	0.8479	–
Upcycled MoE, after 1 epoch (300k jets)	0.8472	0.9852

Table 2: Accuracy before and after upcycling. Training cross-entropy (with label smoothing) over the epoch was 0.828. The model has 9,458,484 parameters.

The MoE keeps the dense accuracy to within 0.15 percentage points. That is the expected outcome of a short fine-tune from an exact copy; it is not evidence of improved tagging.

9.2 Mutual information across layers

The analysis used 20,000 validation jets with 782,261 particles.

Layer	$\bar{H}$ (nats)	type	charge	$\lvert\tanh d_0\rvert$	$\ln p_T$	ΔR^{nn}	jet class
0	2.072	**0.432**	**0.455**	**0.317**	0.068	0.024	0.015
1	2.075	0.130	0.084	0.082	0.070	0.048	0.033
2	2.075	0.095	0.067	0.060	0.026	0.027	0.020
3	2.074	0.184	0.122	0.094	0.066	0.022	0.018
4	2.075	0.085	0.044	0.054	0.044	0.037	0.054
5	2.075	0.130	0.079	0.058	0.052	0.025	0.037
6	2.074	0.044	0.023	0.024	0.029	0.032	0.056
7	2.075	0.051	0.012	0.017	0.033	0.047	0.043

Table 3: Mean router entropy (maximum $\ln 8 \approx 2.079$) and normalised mutual information between the top-1 expert and selected variables, per layer.

Two features stand out (Table 3, Figure 2):

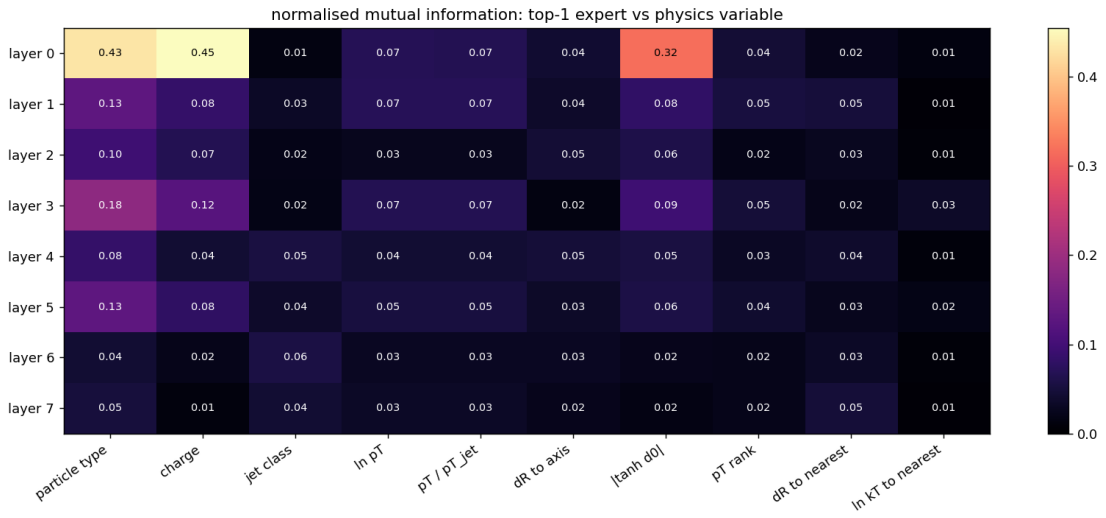


Figure 2: Normalised mutual information between the top-1 expert and each variable, per layer (`analysis/moe8_short/mutual_information.png`).

1. **Layer 0 partitions particles by charge and species.** The NMI with charge is 0.45 and with particle type 0.43, with $|\tanh d_0|$ at 0.32. Kinematic variables are below 0.07. The first router therefore mostly reads particle identity and track displacement, which are explicit input features.
2. **Deeper layers are nearly uniform.** Router entropy is 2.072–2.075 nats in every layer, essentially the maximum of 2.079. Outside layer 0 the largest NMI is 0.18 (particle type in layer 3), and in layers 4, 6 and 7 no value exceeds 0.09. In layers 6 and 7, jet class and nearest-neighbour distance are as informative as type, but all values are small.

Loads are balanced: in every layer each expert receives between 8.2 % and 17.3 % of top-1 assignments.

9.3 Layer-0 experts

Expert	Load	Most enriched type	Charged	$\overline{\ln p_T}$	$\overline{\lvert\tanh d_0\rvert}$	$\overline{\Delta R}$	$\overline{\text{rank}}$
0	13.4 %	ch. had (1.8×)	0.86	1.75	0.024	0.22	23.5
1	11.9 %	muon (4.5×)	0.89	2.02	**0.370**	0.26	20.2
2	12.9 %	photon (2.2×)	0.00	1.27	0.000	0.35	28.9
3	13.0 %	ch. had (2.0×)	0.98	2.65	0.017	0.24	15.1
4	13.1 %	neu. had (3.3×)	0.56	2.07	0.013	0.18	19.1
5	14.4 %	electron (2.5×)	0.37	1.77	0.012	0.27	25.8
6	11.3 %	photon (2.2×)	0.08	1.35	0.002	0.14	27.9
7	10.0 %	neu. had (2.0×)	0.03	2.44	0.012	0.16	14.4

Table 4: Layer-0 expert profiles from `report.json`: top-1 load, the particle type with the largest enrichment $P(e \mid \tau)/\text{load}_e$, charged fraction, and mean $\ln p_T$, $|\tanh d_0|$, ΔR to the jet axis and p_T rank of the particles routed to the expert.

Table 4 and Figure 3 give a readable picture of the first layer. Experts 2 and 6 take photons: expert 2 the softest, widest-angle ones (mean $\ln p_T = 1.27$, $\Delta R = 0.35$), expert 6 photons closer to the axis with small nearest-neighbour k_T, consistent with collinear photon pairs. Experts 4 and 7 are neutral-hadron experts, and expert 3 takes the hardest charged hadrons (mean rank 15, $\ln p_T = 2.65$). Expert 1 is the clearest case: it is 4.5× enriched in muons, its particles have by far

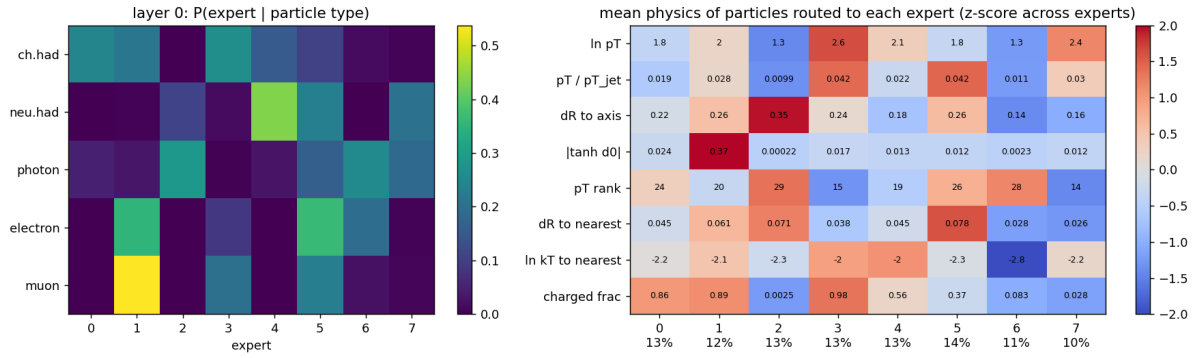


Figure 3: Layer 0: $P(\text{expert} \mid \text{particle type})$ (left) and the mean physics of the particles routed to each expert, as a z-score across experts (right).

the largest mean $|\tanh d_0|$ (0.37, against at most 0.024 for the others), and 53.8 % of all muons go to it. It is also the expert most used by Hbb jets: $P(e = 1 \mid \text{Hbb}) = 0.237$ against 0.075 for QCD. Displaced tracks and soft muons are the textbook signatures of b-hadron decays, so this is the physically expected expert, found without supervision. Its usage is also high in Tbl jets (0.205), which contain a b quark and a lepton.

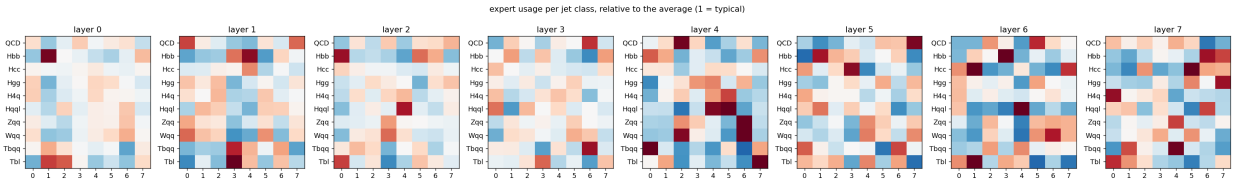


Figure 4: Expert usage per jet class, relative to the average usage, for all eight layers.

9.4 Knockout

The knockout results in `report.json` contain base accuracies for three classes only (QCD 0.764, Hbb 0.916, Hcc 0.842): the knockout pass used 5,000 jets taken in cache order, which covers only these classes. All changes lie between -0.2 and $+0.2$ percentage points. The changes are multiples of $1/2048$, $1/1955$ and $1/997$ for the three classes, so each step is a single jet; for example, removing layer-0 expert 3 costs Hcc 0.2 pp, i.e. two jets. These effects are within statistical noise. With two experts per particle and near-uniform routers, the remaining experts (which are still close copies of the same dense FFN) can absorb the rerouted particles.

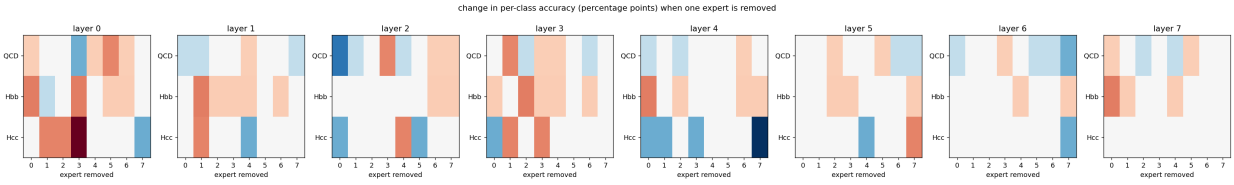


Figure 5: Change in per-class accuracy (percentage points) when one expert is removed.

10 Testing

`tests/test_moe.py` runs on the CPU in a few seconds with a small model ($D = 32$, four heads, three layers, $D_{\text{ff}} = 64$, four experts, top-2, no dropout) and synthetic jets:

- `test_upcycle_matches_dense`: an upcycled MoE with $\sigma = 0$ reproduces the dense logits to 10^{-5}.
- `test_padding_and_aux`: changing padded features does not change the output; recorded routing covers exactly the real particles; `moe_layers=[1,2]` creates MoE only there; the auxiliary loss is positive.
- `test_train_step_and_knockout`: one AdamW step gives every router a non-zero gradient; disabled experts are never selected.

11 Limitations

- **Very short fine-tune.** One epoch on 300k jets is far from convergence of the expert split. The near-maximal router entropy in layers 1–7 shows that the upcycled experts have not yet diverged; conclusions about deeper-layer physics are premature.
- **One seed.** Expert indices are arbitrary and the partition may change with the seed. No second run exists yet.
- **Correlation, not mechanism.** NMI and enrichment are correlations. Particle type, charge and d_0 are direct inputs, so a layer-0 split along them is the easiest thing for a linear router to learn. Kinematic variables are themselves correlated with species.
- **Top-1 summaries of top-2 routing.** The statistics use only the top-1 expert, while the model mixes two experts; a near-uniform π can still yield structured top-1 labels.
- **Small knockout sample.** The stored knockout covers three classes and changes of one or two jets. It cannot yet separate real dependence from noise.
- **Auxiliary-loss scaling.** Balance and z losses are summed over layers and computed per micro-batch, so their effective strength depends on the number of MoE layers and on the micro-batch size.
- **No efficiency gain claimed.** The experts are evaluated in a Python loop with gather/scatter; the implementation targets interpretability, not throughput.

12 Outlook

- Fine-tune for three or more epochs on the full 10M-jet cache, so that the deeper experts have a chance to specialise.
- Sweep `-aux_weight` 10^{-3}, `-top_k 1` and `-moe_layers 4,5,6,7` to obtain sharper, easier-to-read routing (keeping in mind the router-gradient caveat of top-1).
- Repeat with a second seed and compare partitions up to permutation of expert indices.
- Run knockout on a class-balanced sample of all ten classes with enough jets for statistically meaningful per-class changes.
- Upcycle the 20-epoch dense checkpoint once it is available, to start from the strongest dense model.
- Look for a depth trend in which routing moves from raw inputs (type, charge) in early layers to jet-substructure roles (Lund-plane region, class) later. A knockout that mainly hurts Hbb/Hcc for an expert with large $|d_0|$ would identify a heavy-flavour expert; one that hurts Tbl/Hqql and is lepton-enriched would identify a lepton expert.

Source map

File	Contents
`moe.py`	`GDSParTMoEConfig`, `MoEFFN`, `GDSParTMoEBlock`, `GDSParTMoE`, `upcycle_state_dict`
`train_moe.py`	data loading, two-group AdamW, warm-up and cosine schedule, AMP, micro-batching, evaluation and checkpoints
`metrics.py`	accuracy, macro OvO AUC, background rejection at fixed signal efficiency
`analyze_experts.py`	routing recording, physics variables, profiles, NMI, knockout, plots and `REPORT.md`
`gds_part_colab.py`	the unchanged GDS-ParT modules
`data/jetclass_cache.py`	JetClass cache, loader and GPU pair features
`tests/test_moe.py`	CPU unit tests
`analysis/moe8_short/`	the analysis of the first run